Desarrollo y evaluación de un Acelerador Criptográfico para el Algoritmo SHA-2 en Hardware Reconfigurable

1st Benjamín Oyarzún Valdebenito
Departamento de Ingeniería Eléctrica
Universidad de Santiago de Chile
Santiago, Chile
benjamin.oyarzun.v@usach.cl

2nd Nicolás Ruminot Ahumada
Departamento de Ingeniería Eléctrica
Universidad de Santiago de Chile
Santiago, Chile
nicolas.ruminot@usach.cl

3rd Christian Fernández-Campusano
Departamento de Ingeniería Eléctrica
Universidad de Santiago de Chile
Santiago, Chile
christian.fernandez@usach.cl

Resumen—Este documento es un modelo e instrucciones para LaTeX. Este y el archivo IEEEtran.cls definen los componentes de su artículo [título, texto, encabezados, etc.]. *CRÍTICO: No utilice símbolos, caracteres especiales, notas al pie de página, ni matemáticas en el Título del Artículo o Resumen.

Index Terms—aceleradores, funciones hash, SHA-2, minería blockchain, FPGA, Bitcoin.

I. Introducción

En los últimos años, la seguridad de la información ha experimentado una transformación radical debido al crecimiento exponencial en el volumen de datos. Dado que la escalabilidad de los recursos computacionales tradicionales no ha avanzado de manera proporcional, optimizar los procesos de encriptación mediante el uso de aceleradores criptográficos compatibles con los principales estándares de seguridad se ha vuelto una prioridad.

Los Algoritmos de Hash Seguro (SHA) constituyen el núcleo de la seguridad en el ecosistema de telecomunicaciones actual, garantizando la integridad de los datos y validando la identidad del emisor mediante secretos compartidos [1]. El Instituto Nacional de Estándares y Tecnología (NIST) estableció esta familia de funciones [2], publicando SHA-1, SHA-2 y SHA-3. Actualmente, SHA-2 es el mecanismo criptográfico más ampliamente adoptado a nivel global, habiendo superado las vulnerabilidades críticas de sus predecesores, como MD5 y SHA-1 [3], y estableciéndose como el mecanismo de hashing preeminente en aplicaciones prácticas [1].

Dentro de esta familia, variantes como SHA-224, SHA-256, SHA-384 y SHA-512 se utilizan ampliamente en diversas aplicaciones genéricas de seguridad. Tradicionalmente, las funciones hash se aplicaban predominantemente en la seguridad de redes a través de Internet, donde los servidores maximizaban el rendimiento utilizando aceleradores por hardware, priorizando el rendimiento sobre la huella de área y el consumo de energía [1]. Hoy en día, las aplicaciones notables incluyen Códigos de Autenticación de Mensajes Basados en Hash (HMAC), Detección y Corrección de Errores (EDAC), Algoritmos de Firma Digital (DSA), Generadores de Números Pseudoaleatorios (PRNG), sistemas RFID y Computación Confiable. Más allá de estas implementaciones estándar, SHA-256 es una función fundamental en la tecnología blockchain, donde opera en su variante de doble aplicación como motor de consenso (Proof-of-Work) para redes como Bitcoin [4]. En PoW real, cada candidato consiste en una cabecera de 80 B con un nonce generado en el dispositivo, y el digest se calcula dos veces (doble SHA-256). Asimismo, SHA-256 se integra a través de contratos precompilados en plataformas descentralizadas como Ethereum [5] para facilitar la interoperabilidad y la verificación criptográfica entre cadenas.

La elección de evaluar SHA-256 en este estudio responde a su ubicuidad en la infraestructura digital contemporánea. Este algoritmo no solo asegura los protocolos de red tradicionales, sino que también sostiene operaciones críticas de alta demanda, como la construcción y verificación de árboles de Merkle —donde cada nodo interno es el hash de dos hijos de 32 B— y la validación en redes IoT, donde los nodos limitados en recursos imponen estrictos requisitos de eficiencia energética [1]. En este contexto, la minería de criptoactivos y la validación distribuida representan cargas de trabajo de escala global donde la eficiencia energética de las operaciones de hash resulta determinante [6]. Para estas cargas de trabajo de mensajes cortos independientes, el throughput de una CPU de propósito general tradicional resulta insuficiente; sin embargo, como demuestran nuestros resultados experimentales, la eficiencia energética neta de una CPU con instrucciones SHA-NI dedicadas alcanza valores altamente competitivos (5.03 J/GB a 4 núcleos y hasta 2.87 J/GB a 12 hilos). La adopción de aceleradores por hardware responde así a la necesidad de maximizar tanto el rendimiento como la eficiencia energética [7].

Si bien las arquitecturas heterogéneas destacan dos enfoques principales —el paralelismo masivo mediante GPUs y la instanciación de circuitos dedicados en FPGAs— las capas de abstracción introducidas por las herramientas modernas de Síntesis de Alto Nivel (HLS)

a menudo producen inferencias de hardware subóptimas en entornos muy restringidos. Para superar esto, el diseño personalizado a Nivel de Transferencia de Registros (RTL) sigue siendo el estándar de oro. La descripción directa de hardware a través de Verilog permite un control determinista a nivel de ciclo sobre el camino crítico y una asignación precisa de recursos, lo cual es esencial para maximizar la eficiencia energética y el rendimiento en cargas de trabajo criptográficas.

En respuesta a esta necesidad de evaluación, este artículo presenta un análisis de rendimiento comparativo del algoritmo SHA-256 implementado en GPU y FPGA, en contraposición con el procesamiento en CPU. El resto de este documento está organizado de la siguiente manera: la Sección II revisa el trabajo relacionado; la Sección III detalla la metodología y el diseño de los aceleradores propuestos; la Sección IV presenta los resultados experimentales de latencia, throughput y eficiencia energética; y la Sección V presenta las conclusiones de la investigación y el trabajo futuro.

II. Trabajo Relacionado

La aceleración por hardware de la familia de funciones hash SHA-2, específicamente la variante SHA-256, ha sido ampliamente investigada para superar los cuellos de botella de rendimiento y mejorar la eficiencia energética en comparación con las plataformas de procesamiento secuencial tradicionales. Como fue clasificado sistemáticamente por Martino y Cilardo [1], las implementaciones de hardware generalmente se dividen en dos categorías principales: arquitecturas de procesadores programables y arquitecturas de aceleradores dedicados (o coprocesadores). El enfoque de coprocesador domina ampliamente la literatura debido a su superior capacidad para optimizar el rendimiento bruto y maximizar la eficiencia de área [1]. Los enfoques recientes del estado del arte pueden analizarse a través de cuatro ejes principales de optimización: técnicas orientadas a alto rendimiento, sistemas versátiles multimodo y co-diseño, técnicas de ahorro de área/energía para entornos restringidos, y arquitecturas redundantes tolerantes a fallos.

II-A. Optimizaciones Arquitectónicas Orientadas al Rendimiento

Para abordar los estrictos requisitos de throughput de aplicaciones de alta demanda como la minería de blockchain y servidores web de alto tráfico, los diseñadores emplean una combinación de desenrollado de bucles (loop unrolling) [8], [9], segmentación de hardware (pipelining) [10], precomputación de variables [11], [12], y reordenamiento espacial [13] [1]. El desenrollado de bucles reduce el número total de iteraciones al calcular múltiples rondas subsiguientes dentro de un solo ciclo de reloj; si bien esto disminuye directamente la latencia, inherentemente aumenta el camino crítico y el área de hardware [8], [9]. Por el contrario, el pipelining distribuye las iteraciones totales a través de múltiples etapas, permitiendo el procesamiento simultáneo de múltiples bloques de mensajes sin aumentar el retraso de un solo cálculo [10]. Variantes avanzadas, como el quasi-pipelining [14], [15], introducen pipelining dentro del núcleo de la ronda de transformación misma al explotar el reenvío interno de datos [1]. Además, los retrasos en el camino crítico frecuentemente se mitigan mediante la precomputación variable —calculando valores antes de que sean estrictamente necesarios— [11], [12] y el reordenamiento espacial [10], [13], que reubica estratégicamente los registros de pipeline en el medio de la ronda de cálculo para equilibrar mitades paralelas.

Aplicando estos principios a diseños prácticos, Li et al. [16] propusieron una arquitectura de alta velocidad basada en un pipeline completo de 64 niveles implementado en un dispositivo Xilinx Virtex-7. Al optimizar los caminos críticos en la función de compresión e integrar la tecnología de Sumador de Ahorro de Acarreo (CSA) en la etapa de expansión de mensajes, el diseño alcanzó una frecuencia operativa de 429 MHz y un rendimiento masivo de 219.65 Gbps [16]. Empujando aún más los límites de rendimiento en silicio, Nannipieri et al. [17] desarrollaron un Motor Hash altamente optimizado para la Iniciativa de Procesadores Europeos (EPI). Sintetizada en una tecnología ASIC TSMC de vanguardia de 7 nm, su implementación de SHA-2-256 alcanzó rendimientos que superan los 33 Gbps con una disipación de potencia estimada extraordinariamente baja de aproximadamente 13.4 mW [17].

II-B. Aceleradores Multimodo y Co-Diseño HW/SW

En contextos que exigen tanto flexibilidad algorítmica como alta eficiencia energética, se han propuesto diseños versátiles que superan significativamente a los procesadores convencionales. Pham et al. [7] desarrollaron un Acelerador Multimodo SHA-2 (MSA) basado en FPGA capaz de calcular múltiples variantes de la función, así como de ejecutar cálculos dobles de SHA-256 en paralelo. Evaluado en una plataforma Xilinx Alveo U280, este acelerador reportó una eficiencia energética de hasta 38.05 Mhps/W, demostrando ser 543.6 veces más eficiente que una CPU Intel i9-10940X del estado del arte [7].

Dada la complejidad de las implementaciones personalizadas en RTL, los enfoques de co-diseño HW/SW aprovechando la Síntesis de Alto Nivel (HLS) han ganado considerable tracción. Kammoun et al. [18] TODO: Referencia [18] (Anonymous) pendiente de completar a mano apuntaron a la plataforma SoC Xilinx Zynq-7000 utilizando Vivado HLS. Al analizar el perfil del código de software y migrar la función Sha256_transform —que consume muchos recursos— a la Lógica Programable de la FPGA a través de una interfaz AXI4-Stream, su arquitectura híbrida logró una ganancia de tiempo de ejecución del 17 % y una reducción del consumo de energía del 73 % en comparación con una implementación de software independiente en el procesador ARM Cortex-A9 [18].

II-C. Optimizaciones Orientadas al Área y Energía para Entornos Restringidos

Para aplicaciones limitadas por estrictos presupuestos de energía y área, como nodos del Internet de las Cosas (IoT) y etiquetas de Identificación por Radiofrecuencia (RFID), el diseño lógico de hardware prioriza la minimización del consumo de recursos sobre el rendimiento absoluto [1]. Para abordar estas restricciones, se adopta ampliamente la técnica de plegado de bucles (loop folding) [19], [20]; este método representa lo opuesto al desenrollado de bucles al dividir la ejecución de una sola iteración a través de múltiples ciclos de reloj, lo que permite la reutilización de bloques funcionales (por ejemplo, sumadores) y reduce drásticamente la huella del circuito [1].

Martino y Cilardo [21] exploraron este escenario al proponer un acelerador SHA-256 altamente configurable para aplicaciones IoT basadas en blockchain, evaluado en una FPGA Kintex UltraScale+. Su trabajo destaca compensaciones arquitectónicas cruciales: mientras que la aplicación del reordenamiento espacial (Reordered_UF1) maximizó la eficiencia del área, una arquitectura ingenua utilizando el desenrollado de bucles demostró ser la configuración más eficiente en el uso de la energía [21]. Apuntando a estos sectores restringidos, Kieu-Do-Nguyen et al. [22] presentaron una arquitectura independiente de la tecnología orientada a plataformas embebidas. Al minimizar la potencia dinámica, su diseño logró una eficiencia energética de 1094.03 Mbps/W en un estado de bajo consumo de 0.072 W (10 MHz), escalando hasta una eficiencia máxima de 5202.77 Mbps/W al operar a 100 MHz. Complementariamente, Hitesh et al. [23] implementaron el algoritmo SHA-256 en una FPGA Spartan-6, empleando la conversión de reloj con compuertas (gated clock) y el intercambio de recursos aritméticos para reducir drásticamente la utilización del área y el consumo de energía.

II-D. Arquitecturas Tolerantes a Fallos y de Redundancia Híbrida

Si bien la energía y el rendimiento son preocupaciones principales, la confiabilidad sigue siendo crítica cuando los aceleradores criptográficos basados en FPGA se implementan en entornos hostiles susceptibles a fallos transitorios. Abordando este problema, Algredo-Badillo et al. [24] propusieron una novedosa arquitectura de hardware de redundancia híbrida SHA-256 orientada a detectar y corregir errores. Su diseño combina de manera inteligente la Redundancia Espacial (Redundancia Modular Dual) y la Redundancia Temporal incrustadas dentro de una estructura de pipeline de 3 etapas. Este enfoque enmascara eficientemente el retraso asociado habitualmente con la redundancia, reportando un rendimiento de 441.72 Mbps mientras garantiza una robusta integridad de los datos [24].

II-E. Motivación para una Arquitectura SHA-256 Dedicada

Aunque las arquitecturas multimodo SHA-2 recientes ofrecen versatilidad, inevitablemente introducen sobrecargas de área y multiplexación. Dado que SHA-256 sigue siendo el estándar dominante para la seguridad IoT y aplicaciones de Proof-of-Work de alto rendimiento, esta investigación se centra estrictamente en un acelerador dedicado de un solo modo. Este enfoque focalizado también responde a consideraciones de eficiencia energética global: en 2023, la criptominería mundial consumió 119.7 millones de MWh [6], junto con 1.850 millones de metros cúbicos de agua y 90.6 millones de toneladas de emisiones de CO_2. Al aislar SHA-256, nuestro diseño de HW dedicado evita las ineficiencias del multimodo. En lugar de depender de abstracciones inferidas por el compilador, aprovechamos optimizaciones personalizadas a Nivel de Transferencia de Registros (RTL) —tales como enrutamiento determinista y equilibrio de pipeline— para maximizar la eficiencia energética y reducir el consumo de recursos de las cargas de trabajo criptográficas.

III. Metodología

Para evaluar exhaustivamente el rendimiento y la eficiencia energética de SHA-256, se establece un marco heterogéneo que compara CPU, GPU y FPGA. En lugar de evaluar únicamente la minería Proof-of-Work (PoW) sintética —donde cada candidato consta de una cabecera de 80 B fija, se ejecuta doble SHA-256 (SHA-256d) y los nonces se generan localmente en el acelerador sin transferencias masivas de datos—, nuestro marco se enfoca en el procesamiento masivo de mensajes cortos independientes (representativos de la verificación de nodos en árboles de Merkle, validación de paquetes en redes IoT y transacciones blockchain). La minería PoW se mantiene como contexto macroscópico de escala de cómputo.

Esta sección formaliza de manera unificada el modelo de carga de trabajo, las métricas y el protocolo experimental, detallando a continuación las particularidades de cada plataforma.

III-A. Workload Unificado y Modelo de Fragmentación

El conjunto de datos de evaluación consiste en un archivo binario continuo de tamaño exacto 2^{30} bytes ($1\,073\,741\,824$ B $\approx$ 1 GiB). Para modelar mensajes independientes, el archivo se divide en:

$$N = \left\lfloor \frac{2^{30}}{C} \right\rfloor \tag{1}$$

fragmentos contiguos e independientes de C bytes. Cada fragmento representa un mensaje completo que genera un digest criptográfico de 32 bytes. Siguiendo la especificación FIPS 180-4 [2], el procesamiento de cada mensaje requiere agregar un byte delimitador 0x80, relleno de ceros y un campo de 64 bits (8 bytes) con la longitud del mensaje en bits (total: $C+9$ bytes mínimos). Por tanto, el número de

bloques de compresión de 64 bytes procesados por cada fragmento es:

$$B = \left\lceil \frac{C+9}{64} \right\rceil \tag{2}$$

La salida de cada corrida consiste en el arreglo contiguo completo de los N digests resultantes ($N \times 32$ bytes en memoria, modo output-mode full por defecto).

El punto de operación principal del estudio es $C = 64$ bytes ($N = 16\,777\,216$ fragmentos, $B = 2$ bloques de compresión por mensaje, payload útil total de 1 073 741 824 bytes y arreglo de salida de 512 MiB). Adicionalmente, para caracterizar el comportamiento frente al tamaño de mensaje, se realiza un barrido paramétrico en CPU y GPU con $C \in \{64\text{ B}, 1\text{ KB}, 4\text{ KB}, 64\text{ KB}, 1\text{ MB}\}$.

III-B. Métricas de Rendimiento y Eficiencia

El marco define las siguientes métricas normalizadas:

- Throughput útil [GB/s]: Tasa de procesamiento del payload real ($N \cdot C$), excluyendo el padding y la longitud de bloque:

$$\text{Throughput} = \frac{N \cdot C}{t \cdot 10^9} \quad [\text{GB/s}] \tag{3}$$

  donde t es el tiempo de ejecución en segundos y 10^9 corresponde a gigabytes decimales (estándar SI).
- Hashes por segundo [hash/s]: Tasa neta de digests calculados:

$$\text{hashes/s} = \frac{N}{t} \tag{4}$$

- Bloques por segundo [bloque/s]: Incorpora B para reflejar el trabajo aritmético real de compresión SHA-256 realizado por la arquitectura:

$$\text{bloques/s} = \frac{N \cdot B}{t} = \text{hashes/s} \cdot B \tag{5}$$

  Nótese que para $C = 64$ B ($B = 2$), la tasa de bloques duplica a la tasa de hashes debido a que el mensaje requiere un segundo bloque dedicado exclusivamente al relleno y longitud.
- Eficiencia energética [J/GB]: Energía consumida por gigabyte útil procesado:

$$\text{J/GB} = \frac{\Delta E}{\text{iteraciones} \cdot (N \cdot C/10^9)} \tag{6}$$

  donde ΔE representa la energía neta (energía bruta bajo carga menos el consumo en reposo integrado durante el intervalo).
- Factor de Aceleración (Speedup): Se establece una única definición de referencia en todo el documento:

$$\text{Speedup} = \frac{T_{\text{plataforma}}}{T_{\text{SHA-NI 4T}}} \tag{7}$$

  anclada en la configuración CPU SHA-NI con 4 hilos E2E sobre el escenario principal ($C = 64$ B, throughput medido de 3.54 GB/s). Cuando resulte ilustrativo para cuantificar la ganancia de las instrucciones criptográficas frente al cómputo de propósito general, se reportará de forma secundaria y explícitamente rotulada la aceleración relativa a la línea base escalar monopróceso (Speedup vs. Escalar 1T).

III-C. Protocolo de Medición

Cada configuración se ejecuta según el siguiente protocolo:

1. Calentamiento: 3 s de ejecución descartada para estabilizar caches y frecuencias.
2. Bucle de medición: Iteraciones consecutivas durante ≥ 15 s por corrida, lo que garantiza que el tiempo total (≥ 75 s por configuración) domine sobre la resolución del temporizador.
3. Repeticiones: 5 corridas independientes; se reporta media $\pm$ desviación estándar.

Telemetría de energía:

- CPU (Intel RAPL): Se utiliza el dominio PKG (intel-rapl:0), que abarca todos los núcleos, la caché L3 y el controlador de memoria integrado. Se mide la energía en reposo durante 10 s previos al benchmark para calcular la energía neta (bruta menos reposo). Los límites de potencia registrados son PL1 = 15 W (TDP) y PL2 = 39 W (pico transitorio).
- GPU (NVIDIA NVML): Muestreo periódico de la potencia instantánea del dispositivo a 10 Hz (intervalo = 100 ms) mediante la interfaz NVML (nvidia-smi). En paralelo, se registra el contador Intel RAPL PKG del host para obtener simultáneamente la métrica de eficiencia energética del dispositivo (solo dispositivo) y la del sistema completo (con host).

Verificación de correctitud: Antes de cada sesión de medición, cada implementación verifica los 4 vectores estándar del NIST (cadena vacía, "abc", vector de 448 bits y 10^6 caracteres 'a'). Adicionalmente, los digests generados para el workload completo se comparan contra hashlib.sha256 de Python; cualquier discrepancia aborta inmediatamente el benchmark.

III-D. Implementaciones de Línea Base (CPU)

La ejecución de SHA-256 en CPU fue evaluada en un procesador Intel Core i7-1255U (Alder Lake, 10 nm), con 10 núcleos (2 P-cores + 8 E-cores, 12 hilos lógicos) y TDP de 15 W. Todos los hilos de benchmark se fijan a E-cores mediante taskset con los identificadores de CPU 4–11 (obtenidos de /sys/devices/cpu_atom/cpus), eliminando la migración de procesos y estabilizando las métricas. El governor del escalado de frecuencia se mantuvo en modo powersave; las frecuencias efectivas bajo carga no fueron fijadas explícitamente.

Se compilaron dos variantes usando GCC 15.2 con -O3:

- Escalar (x86-64): Instrucciones aritméticas y lógicas estándar, representando el rendimiento puro sin asistencia criptográfica.
- SHA-NI: Instrucciones nativas Intel SHA Extensions (-msha, -msse4.1), que ejecutan las operaciones de

compresión SHA-256 directamente en hardware a través de registros SSE de 128 bits.

El escalado con hilos se evaluó con 1, 2, 4, 8 y 12 hilos (12T utiliza los 8 E-cores disponibles más 4 hilos adicionales sujetos a la política de scheduling del kernel).

III-E. Ejecución Masivamente Paralela (GPU)

La implementación GPU se desplegó en una NVIDIA GeForce RTX 2050 (GA107M, Ampere, sm_86, 16 SMs, 2048 núcleos CUDA, 4 GB GDDR6 a 96 GB/s, TDP 30 W), compilada con nvcc -O3 -arch=sm_86. El throughput se reporta en dos niveles:

- Kernel-only: Datos ya residentes en VRAM; mide el rendimiento puro del cómputo SHA-256 en los SMs.
- E2E (extremo a extremo): Incluye transferencia H2D del payload y D2H de los digests a través del bus PCIe, además de la ejecución del kernel.

El staging de memoria se evalúa en dos modos: pinned (cudaMallocHost, DMA directo) y pageable (malloc, copia adicional a un buffer interno del driver).

III-E1. Kernel Base (v1): El kernel v1 asigna un hilo CUDA por fragmento. Su arreglo de expansión de mensajes estándar w[64] consume ≈100 registros/hilo, limitando la ocupación a 2 bloques residentes por SM (33.3 % de los 1 536 hilos/SM máximos según el análisis ptxas).

III-E2. Kernel de Alta Utilización (v2): El kernel v2 aplica ocho optimizaciones microarquitectónicas para elevar la ocupación SM y el throughput de instrucciones:

1. Reemplazo del arreglo w[64] por una ventana deslizante circular w[16], reduciendo registros vivos de ≈100 a ≤48 (–maxrregcount=48). Esto eleva los bloques residentes por SM de 2 a 5 (ocupación teórica: 83.3 %).
2. Tabla de constantes SHA-256 en memoria ___constant___ (acceso por difusión de warp).
3. #pragma unroll en las 64 rondas de compresión.
4. Instrucción ___byte_perm() para carga big-endian en una sola instrucción PTX.
5. Bloque de relleno (padding) computado en línea, sin derrame a memoria local.
6. Bucle grid-stride para mantener saturados los 16 SMs independientemente de N.
7. Memoria host pinned (cudaMallocHost) para DMA asíncrono.
8. Pipeline triple con streams CUDA que superpone H2D, kernel y D2H en motores DMA independientes.

III-F. Diseño de Microarquitectura RTL

III-G. Optimizaciones de Ruta de Datos y Ruta de Control

IV. Resultados

Esta sección presenta los resultados experimentales obtenidos en las plataformas CPU y GPU. Todas las mediciones corresponden al workload descrito en la Sección III-A: un archivo de 2^{30} B dividido en fragmentos de C bytes; el escenario principal utiliza $C = 64$ B ($N = 16\,777\,216$, $B = 2$). Se realizaron 5 corridas por configuración con bucles $\geq$ 15 s; los valores reportados son media $\pm$ desviación estándar. El speedup se define en relación con la configuración SHA-NI 4T E2E ($C = 64$ B, throughput medido de 3.54 GB/s) como referencia principal; se reporta además el speedup respecto a Escalar 1T como referencia secundaria.

IV-A. Resultados de CPU

La Tabla I presenta los resultados de throughput, hashes/s y speedup para la línea base de CPU en un Intel Core i7-1255U (Alder Lake), medidos sobre 5 corridas con hilos fijados a E-cores (CPUs 4–11). El governor del sistema operó en modo powersave; el muestreo continuo de telemetría bajo carga durante los bucles de $\geq$ 15 s registró frecuencias efectivas de ≈3 483–3 487 MHz para 1T y ≈3 472–3 474 MHz para 2T/4T. Al activar todos los núcleos en 8T y 12T, la frecuencia media se ajustó dinámicamente a 2 860 MHz y 2 450 MHz respectivamente, debido a la restricción térmica y al techo de potencia del paquete PL1 (15 W nominal, disipando ≈20.0–20.7 W bajo carga sostenida). La energía se capturó mediante Intel RAPL en el dominio PKG (intel-rapl:0).

Cuadro I
Resultados de CPU SHA-256 (i7-1255U, E-cores, $C = 64$ B, $N = 16\,777\,216$, $B = 2$)

Config.	Throughput (GB/s)	Mhash/s	J/GB	Speedup[a]	Speedup[b]
Escalar 1T	0.11 ± 0.00	1.71	65.11	0.03×	1.00×
SHA-NI 1T	0.89 ± 0.00	13.90	6.83	0.25×	8.13×
SHA-NI 2T	1.78 ± 0.00	27.78	5.16	0.50×	16.25×
SHA-NI 4T	3.54 ± 0.01	55.34	5.03	1.00×	32.38×
SHA-NI 8T	5.76 ± 0.08	90.07	3.07	1.63×	52.69×
SHA-NI 12T	5.94 ± 0.02	92.86	2.87	1.68×	54.33×

[a]Referencia principal: SHA-NI 4T E2E. [b]Referencia secundaria: Escalar 1T.

La implementación escalar exhibe un throughput de 0.11 GB/s (1.71 Mhash/s), estableciendo la línea base sin asistencia criptográfica y demandando 65.11 J/GB netos (92.72 J/GB brutos a 9.09 W). La variante SHA-NI de un solo hilo logra un throughput de 0.89 GB/s (13.90 Mhash/s) con un speedup de 8.13× sobre el escalar, reduciendo el consumo energético a 6.83 J/GB netos y evidenciando la alta eficiencia del pipeline de instrucciones SHA-256 dedicado. El escalado multihilo es prácticamente lineal hasta 4T: SHA-NI 2T alcanza 1.78 GB/s (27.78 Mhash/s, eficiencia 100 % respecto a 1T), y SHA-NI 4T obtiene 3.54 GB/s (55.34 Mhash/s, eficiencia 99.5 % respecto a 1T, 5.03 J/GB netos). A partir de 8T, el escalado presenta rendimientos decrecientes: SHA-NI 8T logra 5.76 GB/s (1.63× vs. 4T) y SHA-NI 12T alcanza 5.94 GB/s (1.68× vs. 4T). Este aplanamiento responde a la reducción de frecuencia operativa inducida por el límite de potencia del paquete (bajada de 3.47 GHz a 2.86 GHz en 8T y a 2.45 GHz en 12T) combinada con la contención en la caché L2 compartida de los clústeres Gracemont y el arbitraje del bus de memoria DRAM. No

obstante, la eficiencia energética mejora monótonamente con el paralelismo en CPU, alcanzando su óptimo en SHA-NI 12T con 2.87 J/GB netos (3.53 J/GB brutos a 20.0 W).

Cuadro II
Barrido de tamaño de fragmento C en CPU SHA-NI 4T

C	N	B	Throughput (GB/s)	Mhash/s	J/GB
64 B	16 777 216	2	3.54 ± 0.01	55.34	5.03
1 KB	1 048 576	17	8.21 ± 0.01	8.01	1.88
4 KB	262 144	65	8.72 ± 0.12	2.13	1.86
64 KB	16 384	1025	8.97 ± 0.01	0.14	1.71
1 MB	1 024	16385	8.99 ± 0.00	0.01	1.72

El barrido de C en SHA-NI 4T (Tabla II) muestra un incremento significativo de throughput útil al aumentar C: de 3.54 GB/s ($C = 64$ B, $B = 2$) a 8.99 GB/s ($C = 1$ MB, $B = 16\,385$), mientras que el consumo energético neto desciende de 5.03 J/GB a 1.72 J/GB. Este comportamiento se explica porque la proporción de bloques de padding respecto a bloques de datos útiles disminuye drásticamente: para $C = 64$ B, el bloque de relleno representa el 50 % del trabajo de compresión ($B = 2$ bloques por 64 B de payload), mientras que para $C \geq 64$ KB la sobrecarga de relleno es inferior al 0.1 %, estabilizando el throughput en torno a ≈8.97–8.99 GB/s.

IV-B. Resultados de GPU

IV-B1. Ablación v1 → v2: La Tabla III compara los kernels v1 (base) y v2 (alta utilización) para todos los tamaños de fragmento evaluados, con staging pinned. El kernel v2 supera ampliamente al v1 en todos los escenarios, con un factor de aceleración kernel-only que oscila entre 1.0× ($C = 1$ MB, donde $N = 1\,024$ fragmentos no saturan los 16 SMs) y 7.3× ($C = 64$ KB, donde v2 alcanza 22.29 GB/s frente a 3.06 GB/s de v1). Para el escenario principal $C = 64$ B, la mejora es de 4.2× (17.40 GB/s frente a 4.11 GB/s).

Cuadro III
Ablación GPU: kernel v1 (base) vs. v2 (alta utilización), staging pinned

C	Kernel	Kernel-only (GB/s)	E2E (GB/s)	Speedup[a]	Mejora v1→v2
64 B	v1	4.11	2.04	0.58×	—
64 B	v2	17.40	5.01	1.42×	4.2×
1 KB	v1	5.63	2.93	0.83×	—
1 KB	v2	13.72	5.82	1.64×	2.4×
4 KB	v1	3.76	2.35	0.66×	—
4 KB	v2	8.24	5.87	1.66×	2.2×
64 KB	v1	3.06	2.06	0.58×	—
64 KB	v2	22.29	5.36	1.51×	7.3×
1 MB	v1	3.34	2.18	0.62×	—
1 MB	v2	3.37	2.45	0.69×	1.0×

[a]Speedup E2E vs. CPU SHA-NI 4T ($C = 64$ B).

El mejor throughput kernel-only del kernel v2 se alcanza en $C = 64$ KB (22.29 GB/s con $N = 16\,384$), seguido de $C = 64$ B (17.40 GB/s con $N = 16\,777\,216$) y $C = 1$ KB (13.72 GB/s con $N = 1\,048\,576$). Estas configuraciones logran la saturación plena de los SMs y la óptima ocupación de registros. El mayor throughput E2E se obtiene en $C = 4$ KB (5.87 GB/s) y $C = 1$ KB (5.82 GB/s), donde el tamaño de lote por stream y la cantidad de fragmentos permiten superponer de manera óptima las transferencias DMA bidireccionales con la ejecución de los bloques en los tres streams CUDA.

La convergencia v1≈v2 observada para $C = 1$ MB (3.34 vs. 3.37 GB/s kernel-only) confirma que con solo $N = 1\,024$ fragmentos, la ocupación interna del SM deja de ser el factor dominante, pasando el cuello de botella al número insuficiente de warps para esconder las latencias de acceso a memoria global.

IV-B2. Impacto del Staging: La Tabla IV evidencia que, para $C = 64$ B, el throughput kernel-only es prácticamente idéntico entre pinned y pageable (17.40 vs. 17.22 GB/s), confirmando que el cómputo puro en VRAM es independiente de la memoria del host. Sin embargo, el throughput E2E experimenta una severa penalización, cayendo de 5.01 GB/s (pinned) a 3.94 GB/s (pageable) —una degradación del 21.4 %— como consecuencia de la copia intermedia síncrona requerida por el driver entre la memoria virtual paginable y el buffer de intercambio pinned antes de iniciar el motor DMA. Asimismo, la relación entre el cómputo puro y el rendimiento E2E (ratio K/E2E) se incrementa de 3.47× en pinned a 4.37× en pageable.

Cuadro IV
Impacto del staging en GPU v2 ($C = 64$ B)

Staging	Kernel-only (GB/s)	E2E (GB/s)	Ratio K/E2E	Speedup[a]
Pinned	17.40 ± 0.23	5.01 ± 0.01	3.47×	1.42×
Pageable	17.22 ± 0.04	3.94 ± 0.01	4.37×	1.11×

[a]Speedup E2E vs. CPU SHA-NI 4T ($C = 64$ B).

IV-B3. Brecha Kernel-only vs. E2E y Enlace PCIe: Para $C = 64$ B con GPU v2 pinned, el throughput kernel-only (17.40 GB/s) es 3.47× superior al throughput E2E (5.01 GB/s). Esta brecha estructural se origina en la saturación del bus PCIe. Mediante la telemetría de nvidia-smi se verificó que el enlace opera en modo PCIe Gen 4 ×4, con un ancho de banda teórico de 7.88 GB/s unidireccional (≈6.4 GB/s efectivos tras la codificación 128b/130b y sobrecarga de tramas TLP). Bajo el modo output-mode full, el retorno de los $N \times 32$ B = 512 MiB de digests a través del bus D2H impone un tráfico bidireccional masivo sumado al gigabyte de payload H2D, convirtiendo el enlace PCIe en el cuello de botella dominante del sistema.

IV-B4. Barrido de C: La Figura 1 muestra la evolución del throughput frente a C para ambos kernels GPU. El kernel v2 domina en todos los tamaños, alcanzando su pico kernel-only en $C = 64$ KB (22.29 GB/s). En términos de throughput E2E, la GPU v2 se mantiene entre 5.01 y 5.87 GB/s para fragmentos de hasta 64 KB, superando en todos los casos el punto de referencia CPU SHA-NI 4T (3.54 GB/s en $C = 64$ B).

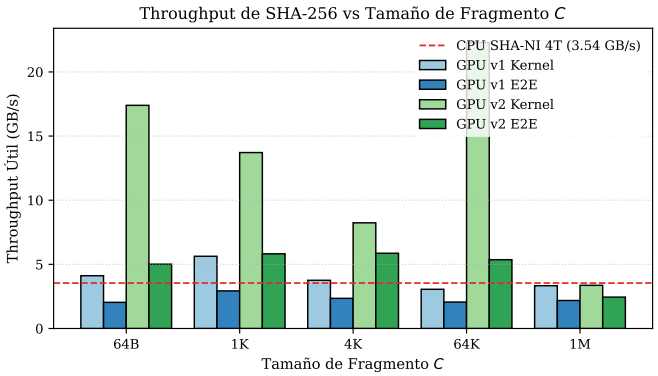


Figura 1. Throughput de SHA-256 vs. tamaño de fragmento C para GPU v1/v2 (kernel-only y E2E, staging pinned) con referencia CPU SHA-NI 4T ($C = 64$ B).

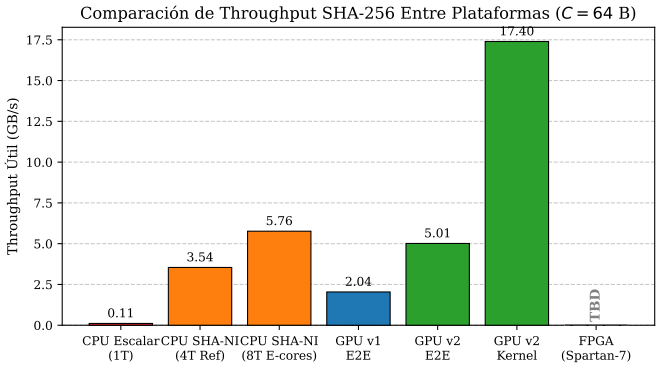


Figura 2. Comparación de throughput útil entre las principales configuraciones de CPU y GPU ($C = 64$ B).

IV-C. Comparación CPU vs. GPU

La Tabla V resume las configuraciones representativas de ambas arquitecturas para el escenario principal $C = 64$ B:

- Rendimiento E2E: La GPU v2 E2E pinned alcanza 5.01 GB/s, superando a la CPU SHA-NI 4T (3.54 GB/s) por un factor de 1.42× (+41.5 %). No obstante, al explotar los 8 E-cores y la concurrencia multihilo en la CPU, SHA-NI 12T alcanza 5.94 GB/s, superando en un 18.6 % al rendimiento E2E de la GPU. La CPU aprovecha el acceso directo a memoria unificada sin la latencia de serialización ni la penalización de transferencia del bus PCIe que restringe a la GPU discreta.
- Potencial de cómputo: El throughput kernel-only de la GPU v2 (17.40 GB/s) es 4.91× superior a la CPU SHA-NI 4T y 2.93× superior a SHA-NI 12T, demostrando que la capacidad aritmética de los 16 SMs supera ampliamente a la CPU, quedando desaprovechada en el entorno E2E debido al cuello de botella de E/S.
- Eficiencia energética: A nivel de dispositivo, la GPU v2 consume 9.69 J/GB (con una potencia media de 17.67 W reportada por NVML); considerando el host completo, el consumo se eleva a 20.17 J/GB. Por su parte, la CPU exhibe una marcada ventaja en eficiencia gracias a la integración nativa de las instrucciones SHA-NI en el silicio del host: SHA-NI 4T demanda 5.03 J/GB netos (6.13 J/GB brutos a 20.8 W), y SHA-NI 12T alcanza 2.87 J/GB netos (3.53 J/GB brutos a 20.0 W). De este modo, para cargas con salida completa de digests, la CPU multihilo es 3.38× más eficiente energéticamente que la GPU a nivel dispositivo, y 7.03× más eficiente considerando la plataforma completa.

Cuadro V
Comparación cruzada de rendimiento y eficiencia energética

Plataforma / Config.	C	Throughput (GB/s)	J/GB	Speedup[a]
Escalar 1T	64 B	0.11	65.11	0.03×
SHA-NI 4T (ref.)	64 B	3.54	5.03	1.00×
SHA-NI 8T	64 B	5.76	3.07	1.63×
SHA-NI 12T	64 B	5.94	2.87	1.68×
GPU v2 kernel	64 B	17.40	9.69	4.91×
GPU v2 E2E pinned	64 B	5.01	9.69	1.42×
FPGA (Spartan-7)	—	TODO:	TODO:	TODO:

[a] Speedup vs. CPU SHA-NI 4T E2E ($C = 64$ B, throughput 3.54 GB/s).

IV-D. Resultados de FPGA

La Tabla VI presentará los resultados post-implementación para el acelerador SHA-256 RTL personalizado sintetizado en la FPGA AMD/Xilinx Spartan-7.

Cuadro VI
Resultados del Acelerador RTL SHA-256 en FPGA (Xilinx Spartan-7)

Parámetro	Valor
Dispositivo Objetivo	Xilinx Spartan-7 (xc7s50)
HDL	Verilog RTL
$f_{máx}$ (MHz)	TBD
Ciclos de Reloj / Bloque	TBD
Throughput (Mbps)	TBD
Utilización de LUT	TBD / 32,600 (TBD %)
Utilización de FF	TBD / 65,200 (TBD %)
DSPs Usados	TBD
Potencia en el chip (W)	TBD
Eficiencia Energética (J/GB)	TBD
Speedup vs. CPU SHA-NI 4T	TBD

La implementación FPGA se enfoca en maximizar la eficiencia energética mediante control RTL determinista: la ruta de datos dedicada elimina la sobrecarga de recuperación de instrucciones y el costo del predictor de saltos inherente a los procesadores de propósito general, a la vez que evita el cuello de botella de la transferencia PCIe que limita el throughput efectivo de la GPU.

Referencias

[1] “martino2020 dummy title,” 2026.
[2] “fips180 dummy title,” 2026.
[3] “sha1 dummy title,” 2026.
[4] “bitcoin dummy title,” 2026.
[5] “eth dummy title,” 2026.
[6] “laimon2025 dummy title,” 2026.
[7] “r1 dummy title,” 2026.
[8] “crowe2004 dummy title,” 2026.
[9] “aisopos2006 dummy title,” 2026.
[10] “michail2009 dummy title,” 2026.
[11] “chaves2006 dummy title,” 2026.
[12] “r3 dummy title,” 2026.
[13] “michail2005 dummy title,” 2026.
[14] “dadda2004 dummy title,” 2026.
[15] “macchetti2005 dummy title,” 2026.
[16] “r2 dummy title,” 2026.
[17] “nannipieri2021 dummy title,” 2026.
[18] Anonymous, “kammoun2020 TODO: completar referencia [18] a mano,” 2026.
[19] “feldhofer2006 dummy title,” 2026.
[20] “kim2008 dummy title,” 2026.
[21] “martino iot2020 dummy title,” 2026.
[22] “r4 dummy title,” 2026.
[23] “hitesh2023 dummy title,” 2026.
[24] “algredo2022 dummy title,” 2026.